

Section 2

2026年1月26日 星期一

01:36

2 Immediate value MOV instructions

Tasks

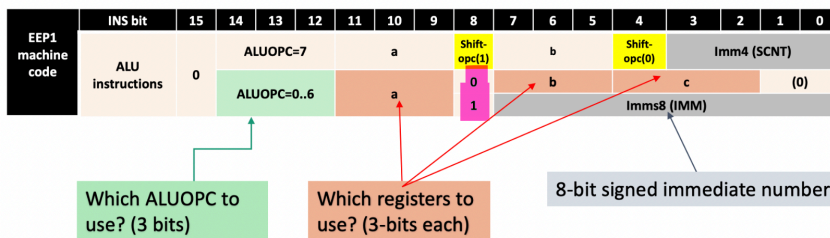
□ **Task 2.** Convert the assembly code in Figure 2 into machine code and load it into *eeep1lab1*.

```
MOV R0, #3
MOV R1, R0
ADD R1, #1
ADD R3, R0, R1
```

Figure 3: Assembly language program with MOV and ADD instructions.

Using [ALU Instructions : detail](#), and [ALU Instruction Encoding](#) sheet, determine which bit(s) in the 16-bit instruction word control:

i. whether a MOV instruction reads from an immediate field (#IMM) or from a register;

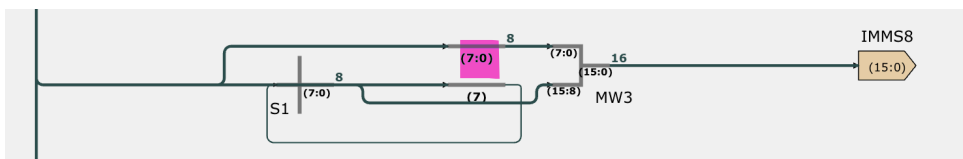


8th bit.

ii. which bits form the 8-bit immediate value.

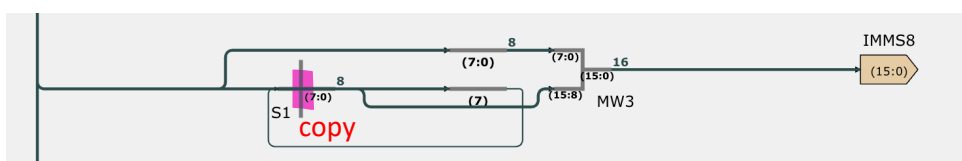
0-7 bits.

3. Open dpdecode and, using your answer to Question ii., investigate through which output port of dpdecode the immediate (#IMM) value is sent in order to reach the ALU's imm port.



It is sent to IMMS8, extends to a 16-bit num, and then inputs into IMM of ALU.

4. Investigate the hardware that converts the 8-bit immediate into the 16-bit value that travels through the ALU and into the register file. How does this hardware sign-extend the 8-bit field to 16 bits so that negative numbers (for example #-3) are preserved when loaded from the instruction word? A good way to answer this is to simulate the instructions in Figure 2 and compare the 8-bit encoded immediate with the 16-bit value observed on the datapath.



It takes INS[7:0] out and extends it to make a 16-bit number
because it is signed number, so doesn't change overall value

because it is signed number, so doesn't change overall value

00000011 -> 00000000 00000011

10000111 -> 11111111 10000111

	0	1	2	3	4	5	0
Datapath.ALU.Imm(15:0)	x0003	x0000	x0001	x002C	x0000	x0000	x0003

This is the extended output.